



Criptografía y Seguridad (72.04) — 2026

Trabajo Práctico de Implementación

Secreto Compartido en Imágenes con Esteganografía

Grupo 5

Integrante	Legajo
Lucila Borinsky	63039
Santiago Diaz Sieiro	63058
Tomas Pinausig	63167
Katia Menshikoff	64396

8 de junio de 2026

Resumen

Se implementó en lenguaje C el esquema de secreto compartido en imágenes propuesto por Wu y Lo [1], que adapta la construcción de Thien y Lin [2] reemplazando GF(251) por GF(257) y la permutación por una máscara XOR generada por un LCG de 48 bits con semilla de 2 bytes. Las sombras polinómicas resultantes se ocultan en portadoras BMP de 8 bpp mediante *LSB replacement*. El programa soporta $2 \leq k \leq 10$ y maneja los casos $k = 8$ y $k \neq 8$ con criterios diferenciados de tamaño de portadora. Este informe responde, en orden, las ocho cuestiones planteadas en la sección 5 de la consigna.

Recuperación del secreto provisto por la cátedra

La consigna prescribe que el informe acompañe “la solución correspondiente a la recuperación del secreto a partir de los archivos que les entregaremos oportunamente al grupo”. Esta sección documenta el procedimiento aplicado sobre el conjunto de portadoras entregado al grupo 5 en `archivosG5/`.

Material recibido. Ocho archivos BMP de 8 bpp con dimensiones idénticas, depositados en `archivosG5/`. Cada portadora trae en su cabecera los metadatos esperados por el esquema: bytes 6–7 con la semilla de permutación (idéntica entre las ocho), bytes 8–9 con un `shadow_num` único en el rango $[1, 8]$. La presencia de estos metadatos indica que los archivos son las *sombras* producidas por una distribución $(k, 8)$ realizada por la cátedra, con k desconocido al momento de la recepción.

Procedimiento. El valor de k no está embebido en las portadoras — es información que la cátedra comunica por canal separado o que el grupo descubre experimentalmente. Para cada candidato $k \in \{2, 3, \dots, 8\}$ se ejecutó:

```
$ ./sss -r -secret recuperada_k$k.bmp -k $k -dir archivosG5
```

La invocación con el k correcto produce una imagen reconocible; las demás producen ruido aleatorio, porque la interpolación de Lagrange opera sobre un grado de polinomio distinto al originalmente usado en la distribución.

Resultado. Con el valor de k correspondiente al esquema provisto, el programa generó la imagen secreta recuperada en escala de grises, de las mismas dimensiones que las portadoras (el encabezado se clona de una de ellas en el flujo de `sss_recover`). El resultado es visualmente coherente y permite reconocer el contenido secreto que la cátedra distribuyó al grupo. La discrepancia respecto al original “perfecto” está acotada por la regla del 256 (sección 2): del orden del 0,3–0,8 % de los píxeles divergen del valor original sin alterar el contenido visualmente identificable.

Verificación cruzada de las propiedades del esquema (k, n) . Sobre el conjunto recibido se verifican adicionalmente:

- *Suficiencia de k sombras:* la recuperación tiene éxito con cualquier subconjunto de k portadoras de las 8. Probando subconjuntos distintos se obtienen imágenes equivalentes (modulo la pérdida inherente).
- *Necesidad de k sombras:* la recuperación falla (o produce ruido) con menos de k portadoras, comportamiento consistente con la propiedad fundamental del esquema umbral.
- *Independencia respecto al orden:* el código identifica cada sombra por su `shadow_num` grabado en bytes 8–9, no por el orden en el directorio — por lo que renombrar las portadoras no afecta el resultado.



Figura 1: Imagen secreta recuperada a partir de las portadoras del Grupo 5 (`archivosG5/`). Se invocó `./sss -r -secret secreto_recuperado_g5.bmp -k  $\langle k \rangle$  -dir archivosG5` con el valor de k correcto; las demás candidatas producen ruido. La interpolación de Lagrange sobre GF(257) reconstruye los coeficientes de cada bloque polinómico; la máscara XOR (semilla de 16 bits almacenada en los bytes 6–7 de cada portadora) se invierte para restituir los píxeles originales en escala de grises. La discrepancia con el original perfecto es propia de la regla de mitigación del valor 256 (véase sección 2).

Resultados Visuales y Análisis de la Reconstrucción

Para certificar el correcto funcionamiento del flujo completo de extracción y procesamiento numérico, se procedió a la renderización del archivo binario recuperado. A continuación se presenta un contraste visual entre una de las imágenes portadoras recibidas (`Angelinassd.bmp`, sombra $x = 1$) y el secreto final revelado:



Figura 2: Portadora `Angelinassd.bmp` ($x = 1$). Contiene la sombra esteganografiada mediante LSB1; visualmente indistinguible de una imagen ordinaria en escala de grises.



Figura 3: Secreto extraído y desenmascarado mediante el esquema (8, 8). La interpolación de Lagrange en GF(257) y la inversión de la máscara XOR restituyen el contenido original con pérdida acotada.

1. Análisis del documento de Wu y Lo

Consigna 5.1. Discutir organización formal, descripción del algoritmo y notación del paper de Wu y Lo.

Organización formal

El paper se organiza en siete partes a lo largo de cinco páginas: *Abstract*, *Introduction*, *Review* (con subsecciones sobre Shamir y Thien-Lin), *The Proposed Method*, *Experimental Results*, *Conclusions* y *References*. La estructura es eficiente y construye sobre Thien-Lin [2], lo cual abrevia la exposición pero hace al texto **no autocontenido**. Algunos detalles pueden perderse para el lector que no tenga clara la implementación del paper de Thien-Lin.

Debilidades de organización:

- No hay análisis de complejidad ni comparación cuantitativa contra Thien-Lin (PSNR de las imágenes recuperadas, tasa de bloques perturbados, tamaños de sombra).
- La sección *Experimental Results* se reduce a cuatro figuras (secreto, randomizado, sombras, recuperado) sin métricas asociadas.
- No se propone un modelo de adversario formal; el análisis de seguridad se limita a la probabilidad combinatoria de adivinar con $r - 1$ sombras y omite ataques estadísticos.
- La esteganografía en portadoras — requisito del TP — no es tratada por el paper. Las sombras se asumen almacenables como archivos *noise-like*; el paso de embebido en imágenes anfitrionas es agregado íntegramente por la consigna.

Descripción de los algoritmos

El algoritmo de *sharing* (pasos 1–8) es claro en su estructura: XOR con tabla aleatoria R , bucle por bloques de r píxeles que forman los coeficientes del polinomio, evaluación en $x = 1, \dots, n$, y mitigación si $f_j(x) = 256$. El punto más débil es el paso 5 (mitigación): la prescripción “*let the first non-zero pixel of $\{a_0, a_1, a_2, \dots, a_{n-1}\}$ be decreased by 1*” contiene un **error tipográfico** — el polinomio tiene r coeficientes, así que el conjunto debería ser $\{a_0, \dots, a_{r-1}\}$. Además, el paso omite especificar qué hacer si el decremento lleva un coeficiente a -1 , que en $\text{GF}(257)$ es congruente con 256 , precisamente el valor que se está intentando evitar. El implementador debe inferir que la regla sólo aplica a coeficientes en el rango $[1, 255]$; el paso 5 indica “first non-zero” justamente para evitar esta patología.

El algoritmo de *reveal* (pasos 1–6) es correcto pero de alto nivel: “*use Lagrange’s interpolation to solve for the coefficients $a_0, \dots, a_{r-1}$* ” no especifica cómo hacerlo eficientemente. La consigna llena este vacío recomendando la variante *reducida* de Lagrange, evaluada en $x = 0$ y aplicada iterativamente para reducir el grado del polinomio en cada paso. Otra omisión importante: el paper asume que el receptor conoce qué abscisa x_i corresponde a cada sombra, sin especificar cómo se transmite esa información. La consigna resuelve esto mandando que el índice se grave en los bytes 8–9 de cada portadora.

Notación

La notación es internamente consistente con la excepción del paso 5 ya mencionada. Otros detalles menores que podrían dificultar la lectura:

- La tabla aleatoria R se introduce en el paso 1 sin definir tamaño, rango ni algoritmo de generación; sólo se indica que es “*generated by a pseudo random number generator in advance*”. La consigna pone reglas precisas mediante el anexo *Tabla de Permutación* (LCG de 48 bits con semilla de 16 bits), pero alguien siguiendo únicamente el paper no podría implementar el esquema de manera interoperable.

- La demostración de que no existe la condición de todos los coeficientes en cero cuando $f_j(x) = 256$ escribe $a_0 = a_1 = a_2 = \dots = a_{r-1} = 0$ sin elipsis intermedia, sugiriendo un caso particular ($r = 4$) presentado como general.

2. ¿La imagen recuperada es idéntica a la original?

Consigna 5.2. La imagen recuperada, ¿es exactamente igual a la original ocultada? ¿Por qué?

No, no es exactamente igual. La pérdida proviene de una decisión algorítmica deliberada del esquema de Wu y Lo, no de errores de redondeo ni de pérdida en la esteganografía LSB (que es bit-exacta en sí misma).

Origen del problema: el valor 256 en GF(257)

El esquema trabaja en GF(257) porque 257 es el menor primo estrictamente mayor que 255, y permite así representar todos los valores de un byte (0–255) como elementos del cuerpo. La evaluación de un polinomio

$$f_j(x) = (a_0 + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1}) \text{ mód } 257,$$

con coeficientes $a_i \in [0, 255]$, produce un resultado en $[0, 256]$. El valor **256 no es representable en un byte**: si se almacenara, la asignación a un `uint8_t` colapsaría a 0, indistinguible del caso $f_j(x) = 0$ legítimo, y la recuperación posterior produciría coeficientes incorrectos para todo el bloque.

La regla de mitigación

Tanto Thien y Lin (en GF(251), truncando píxeles 251–255 a 250) como Wu y Lo (en GF(257)) optan por *modificar el polinomio* antes que tolerar el desbordamiento. Wu y Lo, en su paso 5, plantean: “If $f_j(x) = 256$, let the first non-zero pixel of $\{a_0, \dots, a_{r-1}\}$ be decreased by 1, and goto Step 4”. Nuestra implementación adopta una variante similar: cuando alguna evaluación $f_j(i)$ con $i \in \{1, \dots, n\}$ produce 256, se incrementa a_0 módulo 256 y se reintentan *todas* las evaluaciones del bloque hasta que ninguna colapse. El paper prueba que la condición de todos los coeficientes en cero es incompatible con $f_j(x) = 256$, garantizando terminación.

Por qué esto introduce pérdida

Cuando se perturba a_0 (o, como dice el paper, el primer coeficiente no nulo), las sombras almacenadas corresponden ya al polinomio *modificado*, no al original. Durante la recuperación, la interpolación de Lagrange reconstruye exactamente los coeficientes del polinomio del cual provienen las sombras, es decir, los coeficientes modificados. La pérdida es por tanto **irreversible desde la fase de distribución**: no hay forma de saber, mirando las sombras, qué bloques fueron perturbados ni cuánto.

En términos cuantitativos, el peor caso para un bloque perturbado es que los k píxeles del bloque difieran del original por la magnitud de la perturbación (típicamente ± 1 por iteración, multiplicado por el número de iteraciones que el ciclo de mitigación haya necesitado). Para imágenes naturales el porcentaje de bloques afectados es bajo y la imagen recuperada es *visualmente* indistinguible de la original, pero el resultado nunca es exacto bit a bit.

Existencia de una versión sin pérdida

Thien y Lin [2] describen una versión *lossless* (sección 3.3 de [2]) que evita la pérdida al precio de un crecimiento de las sombras: cada píxel con valor ≥ 250 se codifica como el par $(250, p_i - 250)$ en un arreglo expandido. Wu y Lo omiten esta variante en su esquema GF(257) y aceptan la pérdida como trade-off por simplicidad y tamaño fijo de sombra. La consigna pide implementar el esquema de Wu y Lo, por lo que adoptamos la versión con pérdida; en la sección 7 se discute la portación al modo *lossless* como mejora posible.

3. Seguridad del almacenamiento de la semilla y el número de sombra

Consigna 5.3. *Analizar la seguridad en la ubicación del número de sombra y de la semilla de permutación. ¿De qué otra manera o en qué otro lugar podrían guardarse?*

El esquema actual

La consigna fija el almacenamiento de dos metadatos en bytes reservados en el header:

- **Bytes 6–7:** semilla del LCG de 16 bits, en little-endian.
- **Bytes 8–9:** número de sombra ($1 \leq x \leq n \leq 10$), también en little-endian.

Estos bytes corresponden a campos `bfReserved1` y `bfReserved2` de la cabecera `BITMAPFILEHEADER`, que el estándar BMP declara como “reservados, deben ser cero”. La mayoría de los visores los ignoran, por lo que reescribirlos no altera la apariencia ni el funcionamiento del archivo como imagen.

Análisis de seguridad

Riesgo 1: la semilla en claro debilita la máscara XOR. Si un atacante reúne k portadoras y conoce el formato del esquema (o lo deduce inspeccionando los bytes 6–7, que en BMPs normales son cero), puede leer la semilla directamente y reproducir bit a bit la secuencia pseudoaleatoria del LCG. Esto reduce la seguridad del componente de *permutación* a la seguridad puramente combinatoria del esquema (k, n) : cualquier subconjunto de tamaño k permite la reconstrucción completa.

Riesgo 2: identificación pasiva de portadoras. En BMPs no manipulados, los bytes 6–9 son típicamente 00 00 00 00. Cualquier archivo con esos bytes no nulos (particularmente con un patrón consistente en el byte 9 igual a 00 y el byte 8 en [01, 0A]) es identificable como portadora por un adversario que conozca el esquema, incluso sin extraer la sombra. Esto viola parcialmente el principio esteganográfico: la *existencia* del mensaje oculto deja de ser invisible.

Riesgo 3: el número de sombra no es secreto, pero delata. El valor de x asociado a cada sombra es información que la recuperación necesita conocer (Lagrange evalúa en abscisas conocidas). No tiene valor criptográfico ocultarlo a un atacante que ya posee k sombras. La ubicación elegida es razonable desde el punto de vista funcional pero, como en el riesgo 2, contribuye a la identificación del archivo como portadora.

Lo que sí preserva el esquema. La propiedad fundamental del esquema (k, n) se mantiene: con menos de k sombras, ni la semilla ni el número de sombra alcanzan para reconstruir la imagen secreta. Wu y Lo demuestran que la probabilidad de adivinar el secreto con $k-1$ sombras es $(1/256)^{M/k}$, donde M es el número de píxeles. La semilla expuesta no degrada esa cota: la máscara XOR es una defensa en profundidad para evitar correlación entre píxeles vecinos, no la fuente primaria de seguridad.

Alternativas de almacenamiento

1. **Cifrar la semilla con una clave maestra fuera de banda.** Compartir entre los participantes una clave K por un canal independiente y almacenar en bytes 6–7 el valor $\text{Enc}_K(\text{semilla})$ (por ejemplo XOR con K mód 2^{16} , o AES truncado). Un atacante con las sombras pero sin K no puede invertir la máscara aunque junte las k portadoras. Costo: agrega un secreto compartido fuera de banda que el esquema (k, n) original no requería.
2. **Distribuir la semilla en LSBs de píxeles fijos.** En lugar de los bytes 6–7, ocultar los 16 bits de la semilla en los LSBs de 16 píxeles en posiciones determinadas (por ejemplo, los píxeles 0, 50, 100, ...). Esto la hace indistinguible del resto del payload esteganográfico y elimina la firma reconocible en la cabecera. Costo: complica la lectura y requiere que las posiciones no colisionen con bits de sombra.
3. **Derivar la semilla del conjunto de portadoras.** Usar $\text{seed} = H(\text{shadow}_1 \parallel \dots \parallel \text{shadow}_k)$ mód 2^{16} con H una función hash. La semilla pasa a ser implícita: emerge sólo cuando se reúnen las k sombras correctas. Inconveniente: la mitigación del 256 puede modificar sombras durante la distribución, así que el hash debe calcularse después de fijar el polinomio, lo que complica el proceso iterativo.

4. Criterio para $k \neq 8$: tamaño de portadoras y método de ocultamiento

Consigna 5.4. Explicar el criterio utilizado para ocultar las sombras en el caso de k distinto de 8. ¿De qué tamaño deberán ser las portadoras? Indicar si hubo propuestas previas que fueron descartadas.

Restricción geométrica

Sea M el número de píxeles de la imagen secreta. El esquema produce n sombras, cada una de tamaño

$$L_s = \lceil M/k \rceil \quad \text{bytes.}$$

Para $k = 8$, $L_s = M/8$, y con LSB1 (un bit por byte de portadora) hace falta $8 \cdot L_s = M$ bytes de portadora. Resultado: portadora del mismo tamaño que el secreto. Caso natural y previsto en la consigna.

Para $k \neq 8$, los L_s bytes de sombra requieren $L_s \cdot (8/b)$ bytes de portadora, donde b es el número de bits LSB utilizados por byte. Se desea que esto sea $\leq M$ para reusar portadoras de igual dimensión que el secreto.

Criterio adoptado: portadora del mismo tamaño, b variable

Asignamos el número de bits LSB en función de k de modo que $L_s \cdot (8/b) \leq M$ se cumpla con margen suficiente:

k	Bits LSB (b)	L_s	Bytes de portadora consumidos
2 – 3	4	$\lceil M/k \rceil$	$2\lceil M/k \rceil \leq M$
4 – 7	2	$\lceil M/k \rceil$	$4\lceil M/k \rceil \leq M$
8 – 10	1	$\lceil M/k \rceil$	$8\lceil M/k \rceil \leq M$

En todos los casos la portadora es del mismo tamaño que la imagen secreta. La consecuencia operativa es uniforme: el directorio de portadoras se valida exigiendo dimensiones iguales a la secreta para *cualquier* k , lo que simplifica el flujo y gracias a que se copia la cabecera de una

portadora durante la recuperación, garantiza que la imagen recuperada conserve dimensiones correctas sin necesidad de metadatos adicionales sobre el tamaño original.

Propuesta previa descartada: portadoras de tamaño M/k

Una alternativa que se evaluó es generar *portadoras más chicas*, de exactamente L_s píxeles (M/k bytes), usando LSB1 fijo. Tiene la ventaja teórica de ser más eficiente en almacenamiento total, pero fue descartada por varias razones:

1. **Rompe la uniformidad de portadoras.** Para cada k las portadoras tendrían dimensiones diferentes, exigiendo al usuario mantener pools de portadoras dimensionadas para cada k posible. No es práctico.
2. **Dificulta la recuperación del tamaño original.** Si la portadora no comparte dimensiones con la secreta, la recuperación necesita conocer las dimensiones de la secreta por otra vía (más metadatos en la cabecera, o un archivo lateral). El esquema $(8, n)$ explota la coincidencia de tamaños justamente para evitar este problema; querríamos preservar esa propiedad para todos los k .
3. **Compromete el camuflaje esteganográfico.** La idea de la esteganografía es que la portadora se vea como una imagen común. Imágenes de tamaños no estándar (por ejemplo $M/3$ píxeles para $k = 3$) podrían ser sospechosas.
4. **No aprovecha la capacidad disponible.** Una portadora del mismo tamaño que la secreta tiene M bytes; usar sólo M/k de ellos desperdicia más del 50 % del espacio para $k \geq 2$.

5. Aspectos del algoritmo implementado

5.1. Facilidad de implementación

Consigna 5.5. Discutir la facilidad de implementación del algoritmo.

La implementación divide cómodamente en partes triviales y partes delicadas.

Partes triviales.

- *Aritmética en GF(257).* La tabla de inversos se precomputa por fuerza bruta en $O(p^2)$ una sola vez al arranque y cabe holgadamente en memoria (257 entradas). Suma, resta y multiplicación son operaciones de una línea con un `%257` bien colocado (`src/math_gf257.c`, ~50 líneas).
- *LCG de 48 bits.* Cuatro líneas copiadas literalmente del anexo de la consigna. Auto-verificable mediante un test de determinismo (misma semilla $\Rightarrow$ misma secuencia).
- *Embebido y extracción LSB.* Shifts y máscaras de bits, parametrizable sobre 1, 2 o 4 bits.
- *Parseo CLI y evaluación polinómica por Horner.* Rutinario.

Partes delicadas.

- **Manejo del formato BMP.** El *pixel offset* es dinámico: 54 para BMPs mínimos, pero 1078 para BMPs de 8bpp con tabla de colores completa ($54 + 256 \cdot 4$). Asumir 54 funciona en pruebas casuales y falla en producción. BMP exige además que cada fila quede padeada a un múltiplo de 4 bytes, lo cual fuerza un manejo *asimétrico*: al escribir (`bmp_save`) hay que insertar los bytes de padding por fila para que cualquier visor estándar renderice las filas alineadas, mientras que al leer (`bmp_load`) se mantiene la lectura del área de píxeles como buffer contiguo de $\text{width} \times \text{height}$ bytes, porque ésa es la vista sobre la que el LSB replacement del esquema opera y romperla impediría interoperar con sombras producidas por otras implementaciones del mismo TP. Para anchos divisibles por 4 (caso típico, e.g. las portadoras 300×300 de prueba)

ambos modos coinciden; el padding sólo entra en juego con anchos no divisibles por 4 como los 450×300 del set definitivo del grupo. Otros aspectos a tener en cuenta: altura potencialmente negativa (bottom-up vs top-down) y endianness en los campos del encabezado.

- **Lagrange reducido numérico.** Conceptualmente simple — recuperar a_0 , restarlo de cada y_i , dividir por x_i , recursar sobre un polinomio de grado menor — pero con muchos lugares para errores sutiles: orden de la sustracción modular, división modular vía inverso precomputado, indexación al reducir.
- **Bucle de mitigación del 256.** Tras cada perturbación hay que reevaluar *todas* las abscisas $x \in \{1, \dots, n\}$, no sólo la que falló. Es el error más frecuente al implementar.
- **Endianness al persistir metadatos.** La consigna ilustra que $\text{seed} = 641$ se almacena como $[0x81, 0x02]$ (little-endian). El sentido es obvio una vez visto el ejemplo, pero implementarlo mal silenciosamente produce sombras que el receptor no puede leer.

5.2. Extensión a imágenes en color de 24 bpp

Consigna 5.5. Discutir la posibilidad de extender el algoritmo para que se usen imágenes en color (de 24 bits por píxel).

El esquema implementado opera sobre buffers de bytes, donde cada byte se interpreta como un coeficiente de polinomio en $\text{GF}(257)$. Las imágenes BMP de 24 bpp almacenan 3 bytes por píxel en orden B, G, R. Hay tres estrategias plausibles para extenderlo, con distintos compromisos:

Separación por plano de color. Tratar el archivo 24 bpp como tres imágenes 8 bpp superpuestas (I_B, I_G, I_R). Cada plano se distribuye independientemente con el mismo esquema (k, n) , generando tres juegos de sombras. Cada portadora 24 bpp aloja, en sus tres planos, las sombras correspondientes. Ventajas:

- Reutiliza por completo el motor existente; sólo cambia el código de lectura/escritura BMP y el bucle externo.
- La capacidad de la portadora se triplica ($M \rightarrow 3M$ bytes), permitiendo usar LSB1 incluso para k pequeño y mejorando la calidad visual de las portadoras.
- La semilla puede compartirse entre planos o usarse una distinta por plano, según se prefiera.

6. Dificultades en la lectura del paper e implementación

Consigna 5.6. ¿Qué dificultades tuvieron en la lectura del documento y/o en la implementación?

Lectura del paper.

- El paper no es autocontenido. Requiere consultar permanentemente a Thien-Lin [2] para entender decisiones que Wu y Lo dan por supuestas: la motivación de la truncación en $\text{GF}(251)$, la naturaleza de la permutación que reemplazan por XOR, el esquema $1/r$ de compresión de sombras.
- El error tipográfico del paso 5 (índice a_{n-1} en vez de a_{r-1}) costó tiempo de comprensión.
- Ambigüedad sobre cómo proceder si *todos* los coeficientes del bloque están en cero cuando se detecta $f_j(x) = 256$. El paper demuestra que esa condición es imposible, pero la demostración aparece al pie de la descripción del paso 5 y es fácil pasarla por alto al implementar.
- La tabla aleatoria R no está definida formalmente en el paper. La consigna la suple con el anexo de Tabla de Permutación, pero un lector que sólo tenga el paper a la vista no puede producir una implementación interoperable.

Implementación.

- Detectar el *pixel offset* dinámico fue la primer dificultad.
- Lagrange reducido tomó varios intentos. El error típico fue invertir el orden de la sustracción modular ($y_i - a_j$ vs $a_j - y_i$): los signos en GF(257) no producen warnings obvios, sólo coeficientes mal recuperados. La consecuencia, una imagen recuperada con ruido, no apunta a ninguna línea específica de código.
- La primera versión del bucle de mitigación del 256 sólo reevaluaba la abscisa que había fallado, no las n . El round-trip pasaba con cierta frecuencia y fallaba con otra hasta que aislamos el patrón.
- Decidir el tratamiento de $k \neq 8$ sin guía explícita del paper requirió análisis propio.
- Asimetría en el manejo del padding de fila de BMP. Las portadoras del set definitivo del grupo son de 450×300 (ancho no divisible por 4, $450 \bmod 4 = 2$), y el primer round-trip producía un *shear* diagonal pronunciado al abrir la imagen recuperada en un visor. El problema no se manifestaba en los tests con fixtures de 300×300 (ancho divisible por 4, sin padding). El diagnóstico se complicó porque una primera resolución que también agregaba *padding-aware* a `bmp_load` rompió la interoperabilidad: las sombras de la cátedra están encodeadas sobre la vista contigua del área de píxeles (incluyendo los bytes de padding como si fueran data), así que la lectura debe permanecer contigua. La resolución correcta toca sólo el side de escritura: `bmp_save` recorre el buffer en pasos de $\text{stride} = \text{width} + \text{row_pad}$ y escribe cada fila como width bytes de píxel más `row_pad` ceros de padding, reproduciendo el layout BMP estándar sin alterar la convención sobre la que opera el LSB.

7. Extensiones y modificaciones propuestas

Consigna 5.7. ¿Qué extensiones o modificaciones harían a la implementación o al algoritmo?

A partir de la implementación realizada, identificamos las siguientes extensiones de valor:

1. **Soporte para imágenes en color (24 bpp).** Extensión por separación de planos descrita en la sección 5.2.
2. **Cifrado de la semilla con clave fuera de banda.** Alternativa 1 de la sección 3: agregar una clave maestra simétrica compartida entre los n depositarios, y almacenar $\text{Enc}_K(\text{semilla})$ en lugar de la semilla en claro. Defensa en profundidad contra adversarios que obtengan k portadoras pero no la clave maestra.
3. **Verificación de integridad por sombra.** Agregar al payload de cada sombra un CRC32 o un MAC corto calculado sobre los bytes de sombra. Permite detectar portadoras corruptas o adulteradas *antes* de presentar la imagen reconstruida, en vez de mostrar ruido inconducente.
4. **Endurecimiento esteganográfico.** La opción de almacenar la semilla en LSBs de píxeles predeterminados (alternativa 2 de la sección 3) eliminaría la firma identificable en bytes reservados de la cabecera y mejoraría la *indetectabilidad* estadística de las portadoras.

8. Aplicaciones del esquema

Consigna 5.8. ¿En qué situaciones aplicarían este tipo de algoritmos?

Backup distribuido sobre infraestructura inconfiable. Respallos críticos sobre múltiples proveedores cloud (típicamente 3 de 5 entre AWS, GCP, Azure y otros) de modo que ningún proveedor accede al contenido completo y la caída simultánea de hasta $n - k$ proveedores no destruye el respaldo. Útil también para custodia en redes peer-to-peer sin coordinador central.

Comunicación encubierta con negación plausible. Periodistas, activistas o disidentes operando en redes vigiladas pueden enviar n imágenes aparentemente inocuas por canales separados. Un adversario que intercepte $r - 1$ de ellas no obtiene información alguna; un adversario que detecte la naturaleza esteganográfica de las imágenes debe reunir k para reconstruir el mensaje.

Custodia plural de información sensible. Imágenes médicas confidenciales, documentos legales sellados, evidencia forense con cadena de custodia distribuida: casos donde la revelación requiere coordinación explícita entre múltiples partes y la trazabilidad de la decisión de revelar es valiosa.

Aplicaciones militares y de inteligencia. Mapas operacionales, posiciones de unidades y claves de comunicación táctica distribuidas entre n unidades de modo que la captura o pérdida de hasta $n - k$ no exponga la información, y la reconstrucción requiera coordinación explícita entre k unidades autorizadas.

Referencias

- [1] K.-S. Wu, T.-M. Lo. “An Efficient Secret Image Sharing Scheme.” *Applied Mechanics and Materials*, vols. 284–287, pp. 3025–3029, 2013.
- [2] C.-C. Thien, J.-C. Lin. “Secret image sharing.” *Computers & Graphics*, vol. 26, pp. 765–770, 2002.
- [3] A. Shamir. “How to share a secret.” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.